WEEK 2
Provider-Agnostic
LLM Client

1. Project Overview

The objective of Week 2 is to build a provider-agnostic Large Language Model (LLM) client. The application can send a prompt to either a cloud-based Gemini model or a locally running Llama 3.2 model through Ollama. The rest of the application uses one common client interface, so the provider can be changed without rewriting the application logic.

2. Objectives

- Understand abstraction and inheritance using Python object-oriented programming.
- Define a common LLM provider interface with an abstract base class.
- Connect to Google Gemini using the Google Gen AI Python SDK.
- Connect to a local Ollama model through its HTTP API.
- Load API credentials securely from a .env file.
- Run the application and verify the local model.
- Test the common client independently using a fake provider.

3. Tools and technologies

Tool / Library	Purpose
Python 3	Programming language
Visual Studio Code	Code editor and integrated terminal
Ollama	Runs the Llama 3.2 model locally
Google Gemini API	Cloud-based LLM provider
google-genai	Python SDK for Gemini
requests	Sends HTTP requests to Ollama
python-dotenv	Loads environment variables from .env
unittest / pytest	Testing tools; the test below uses unittest

4. Project Folder Structure

The project folder is named WEEK2. It contains the following files and folders:

```
WEEK2/
├── venv/                # Python virtual environment
├── .env                 # Gemini API key (kept private)
├── llm_client.py        # Provider classes and common client
├── main.py              # Program entry point
├── requirements.txt     # Python dependencies
└── test_client.py       # Unit test for LLMClient
```

The __pycache__ folder may also appear automatically after Python runs. It stores compiled Python files and does not need to be edited.

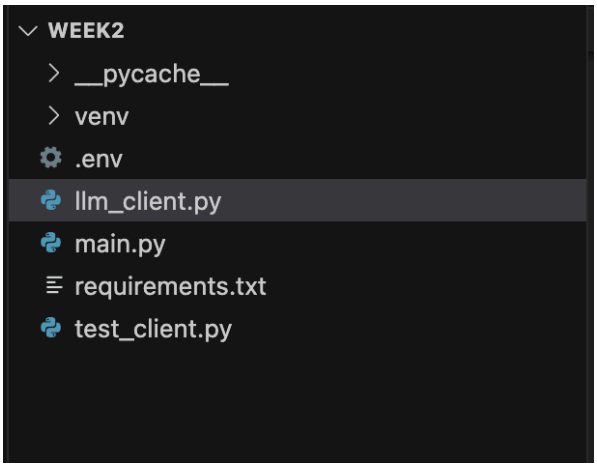


Figure 1. WEEK2 project folder and files in Visual Studio Code.

5. Environment Setup on macOS

1. Open the WEEK2 folder in Visual Studio Code.
2. Open the integrated terminal using Terminal → New Terminal.
3. Create a virtual environment (if it has not already been created): python3 -m venv venv
4. Activate it: source venv/bin/activate. The terminal should show (venv).
5. Install dependencies: pip install -r requirements.txt
6. Install and start Ollama, then download the model using: ollama pull llama3.2
7. The Ollama service should be running locally. The application sends requests to http://localhost:11434.

6. Configuration: .env

The Gemini API key is stored in a .env file so that it is not hard-coded into the Python source. Replace the placeholder below with the API key created for your Google AI Studio project.

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

7. Dependencies: requirements.txt

```
python-dotenv
pytest
google-genai
requests
```

Install all listed packages from the project terminal with:

```
pip install -r requirements.txt
```

8. Source Code

8.1 llm_client.py

This file defines the abstract provider interface, the Gemini implementation, the Ollama implementation, and the common LLMClient. Both providers implement the same generate() method.

```
from abc import ABC, abstractmethod
from google import genai
import requests


class LLMProvider(ABC):
    @abstractmethod
    def generate(self, prompt):
        pass


class GeminiProvider(LLMProvider):
    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)

    def generate(self, prompt):
        response = self.client.models.generate_content(
            model="gemini-3.5-flash",
            contents=prompt
        )
        return response.text


class OllamaProvider(LLMProvider):
    def __init__(self, model="llama3.2"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
        response = requests.post(
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=120
        )
        response.raise_for_status()
        return response.json()["response"]


class LLMClient:
    def __init__(self, provider):
        self.provider = provider

    def generate(self, prompt):
        return self.provider.generate(prompt)
```

8.2 main.py

The main program loads the .env file, asks the user to select a provider, creates the selected provider object, and sends the prompt through the shared LLMClient.

```
import os
from dotenv import load_dotenv
from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

load_dotenv(override=True)

provider_name = input(
    "Choose provider (gemini/ollama): "
).strip().lower()

if provider_name == "gemini":
    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise ValueError(
            "Gemini API key not found in .env"
        )

    provider = GeminiProvider(api_key)

elif provider_name == "ollama":
    provider = OllamaProvider("llama3.2")

else:
    raise ValueError("Invalid provider selected")

client = LLMClient(provider)

prompt = input("Enter your question: ")
answer = client.generate(prompt)

print("\nLLM Response:")
print(answer)
```

8.3 test_client.py

This unit test uses a FakeProvider instead of calling a real AI service. It checks that LLMClient forwards the prompt to its provider and returns the provider's response.

```
import unittest
from llm_client import LLMClient


class FakeProvider:
    def generate(self, prompt):
        return "AI stands for Artificial Intelligence."


class TestLLMClient(unittest.TestCase):
    def test_generate(self):
        provider = FakeProvider()
        client = LLMClient(provider)

        result = client.generate("Explain AI")

        self.assertEqual(
            result,
            "AI stands for Artificial Intelligence."
        )


if __name__ == "__main__":
    unittest.main()
```

9. Running the Application

Run these commands in the VS Code terminal from inside the WEEK2 folder:

```
source venv/bin/activate
python3 main.py
```

When prompted, enter one of the following provider choices:

- ollama – uses the local Llama 3.2 model.
- gemini – uses the Gemini cloud API and the key stored in .env.

Then type a question at the Enter your question prompt. The generated answer is displayed under LLM Response.

The following screenshot records a successful execution using Ollama. The terminal shows the virtual environment activated, main.py running, the ollama provider selected, the prompt “what is ai?”, and a generated response.

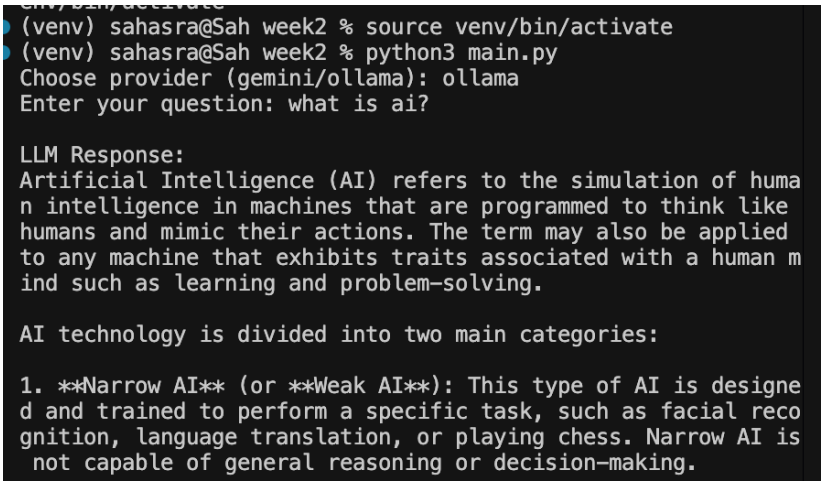


Figure 2. Successful Ollama execution with the Llama 3.2 model.

10. Result

The provider-agnostic LLM client was implemented using Python OOP. The application successfully ran with the local Ollama Llama 3.2 provider and generated an answer to the prompt “what is ai?”. The code also includes the Gemini provider, environment-based API-key configuration, and a unit test for the common LLMClient interface.

11. Conclusion

This project demonstrates how a single application can support multiple LLM providers through a common abstraction. By separating provider-specific code from the client logic, the design is easier to maintain and extend. Another provider can be added by implementing the LLMProvider interface without changing how the main application calls generate().